CS 499 Milestone Two Narrative
Enhancement One: Software Design and Engineering

By John Grudzinski

Artifact for CS 360 Android Inventory App

Brief Description of the Artifact

The artifact I selected for first enhancement is my Android inventory management application that I created for CS 360. The original application allowed a user to create an account, log in, manage inventory items stored in SQLite, view inventory in a RecyclerView, add and delete items, increase or decrease item quantities, and optionally receive SMS alerts when stock was low or out of stock. The project included multiple Java classes, including LoginActivity, MainActivity, DatabaseHelper, InventoryAdapter, InventoryItem, SMSPermissionActivity, and SmsUtils.

Reason for Selecting the Artifact

I selected this artifact because it is a strong representation for the software design and engineering category. It includes several important parts of a realistic application such as a user interface, persistent storage, authentication, CRUD operations, runtime permission handling, and communication between multiple classes. It also connects well to my target specialization of secure full stack software development. Although this is a mobile application rather than a web application, it still demonstrates many full stack concepts because it includes presentation logic, business logic, data storage, and security concerns.

The original version of the application already demonstrated some of my practical Android development skills, but it also had clear areas where the design could be improved. For example, passwords were stored in plain text, inventory records were not tied to specific users, validation was limited, some listener logic was repeated, and some SMS related logic was

duplicated between classes. These weaknesses made the artifact a good candidate for enhancement because they allowed me to show growth beyond simply having a working application.

Enhancements Performed

The first major enhancement I made was improving authentication security. In my original code, user passwords were stored directly in the SQLite users table. The way it works now in the enhanced version is that DatabaseHelper now stores a salted PBKDF2 password hash instead of storing the original password. The createUser method generates a unique salt, hashes the password, and stores the hash and salt. The validateUser method retrieves the stored salt and hash, hashes the entered password, and compares the values using a constant-time comparison. This change directly improves the security of the application and shows a greater understanding of secure design practices.

The second enhancement I made was adding user specific inventory records. In my original version, the application had login functionality, but the inventory table did not include a user identifier. Due to this prior limitation, inventory behaved like one shared list instead of separate user data. In the new enhanced version, the items table includes a user_id column, and the login session stores the current user's database ID. MainActivity now loads, adds, updates, and deletes inventory records using that user ID. This improves privacy and makes the application behave more like a realistic system with multiple users.

The third enhancement was improving input validation. The original add-item dialog checked for an empty item name, but invalid quantity input could silently default to zero which was problematic. In the enhanced version, the dialog keeps the user on the form until valid information is provided. It rejects empty item names, item names that are too long, missing

quantities, non-numeric quantities, and negative quantities. This improves the user experience and prevents bad data from reaching the database.

The fourth enhancement was improving maintainability by reducing repeated logic. In the original MainActivity, the adapter listener setup was repeated in both onCreate and reloadFromDb. In the enhanced version, that listener setup was moved into a single configureAdapterListeners method. This makes the code easier to maintain because the delete, increment, and decrement behavior now has one central location.

The fifth enhancement was focused on centralizing SMS-related logic. SmsUtils already existed in my original project, but the enhanced version expands its role. It now centralizes SMS permission checks, emulator detection, saved destination retrieval, default destination handling, and the actual SMS send operation. MainActivity and SMSPermissionActivity now call these shared helper methods instead of duplicating similar logic. This improves structure, readability, and separation of concerns.

The sixth enhancement was concerned with improving database version handling. In my original project, onUpgrade method was only a placeholder. In my new enhanced version, the database version was increased and onUpgrade includes migration logic for the new password hash, password salt, and user_id fields. This demonstrates more realistic schema management and shows that I considered how an existing database would change as the application improved.

Skills Demonstrated

This enhancement showcases several software design and engineering skills. I demonstrated secure authentication design through password hashing and salting. I demonstrated database design by changing the SQLite schema to support user-specific inventory records. I demonstrated maintainability by refactoring duplicated listener behavior into a reusable method. I demonstrated defensive programming by strengthening input validation and checking

ownership of records before update or delete operations. I also demonstrated modularity by moving shared SMS functionality into SmsUtils rather than leaving repeated logic spread across multiple activities.

The enhanced artifact also shows my ability to improve existing software instead of only building from scratch. This is important because real software development often involves analyzing older code, identifying weaknesses, and making targeted improvements without breaking the user's existing workflow. I kept the original purpose of the application intact while improving its security, organization, and reliability.

Course Outcomes Addressed

The enhancements I've made support Course Outcome 4, which focuses on using well founded techniques, skills, and tools in computing practices to implement solutions that deliver value and accomplish industry specific goals. The enhanced Android app uses Java, Android Studio, SQLite, RecyclerView, runtime permissions, and structured helper classes to create a more realistic and valuable inventory management tool.

This enhancement also supports Course Outcome 5, which focuses on developing a security mindset. The original application stored plain text passwords and did not separate inventory records by user. The enhanced version mitigates those design flaws by adding salted password hashing and user specific data access. These changes show that I considered privacy, data protection, and potential misuse of application data.

This milestone also supports Course Outcome 2 because the accompanying narrative and code comments explain the changes clearly for both my academic and professional audience. My main outcome coverage for this milestone is focused on software design and security. I still plan to demonstrate Course Outcome 3 more strongly in the algorithms and data structures enhancement and Course Outcome 1 more strongly in the database/dashboard enhancement.

Reflection on the Enhancement Process

The enhancement process helped me better understand how software design decisions affect security, maintainability, and future growth. One challenge was improving authentication without changing the entire login workflow. I wanted the application to feel the same to the user, but work more securely behind the scenes. Adding salted password hashing helped me meet that goal because the UI remains familiar while the data storage is much safer.

Another challenge was adding user specific inventory without overcomplicating the app. The original application already stored a username in SharedPreferences, but that alone was not enough to separate data. I needed to store and use the database user_id so inventory records could be filtered by owner. This required changes in DatabaseHelper, LoginActivity, and MainActivity. It also helped me see how a design change in one part of an application can require coordinated changes across several classes.

Refactoring the adapter listener logic also reinforced the importance of avoiding repeated code. The original repeated listener setup worked, but it would have made future changes harder. Moving the repeated behavior into configureAdapterListeners made the code cleaner and easier to understand. This is a practical example of improving maintainability without changing the main user facing behavior.

Overall, this enhancement improved the artifact from a functional class project into a stronger portfolio artifact. The application now demonstrates more professional software engineering practices, including secure authentication, user data separation, validation, database migration, and reusable helper logic. These improvements align with my goal of presenting myself as a secure full stack software developer who can analyze existing code and improve it in meaningful ways.